

MODUL VI INTERRUPT DAN TIMER

Ahmadin Irfan Fauzi (H1H024060)

Asisten: Athallah Tsany Satriyaji

Tanggal Percobaan: 13/05/2026

TK244005-Praktikum Sistem Mikrokontroler

Laboratorium Multimedia – Jurusan Informatika UNSOED

Abstrak

Praktikum ini bertujuan untuk mempelajari dan mengimplementasikan mekanisme interrupt dan timer pada mikrokontroler Arduino Uno. Percobaan 6A mengimplementasikan external interrupt menggunakan push button pada pin 2 untuk mengendalikan kondisi LED secara toggle melalui Interrupt Service Routine (ISR) dengan mode pemacu FALLING, sehingga sistem tidak memerlukan polling tombol secara terus-menerus. Percobaan 6B mengimplementasikan timer berbasis fungsi millis() untuk mengedipkan dua LED dengan frekuensi berbeda secara non-blocking, memungkinkan program tetap responsif selama LED berkedip. Hasil percobaan menunjukkan bahwa interrupt dan millis() mampu meningkatkan efisiensi CPU dan responsivitas sistem secara signifikan dibandingkan metode polling dan delay() konvensional.

Kata kunci: Arduino Uno, interrupt, ISR, timer, millis(), non-blocking, LED.

1. PENDAHULUAN

Mikrokontroler seperti Arduino Uno pada dasarnya menjalankan instruksi secara sequential dari awal hingga akhir. Pada sistem sederhana, pengecekan kondisi input dilakukan secara polling — CPU terus-menerus memeriksa kondisi pin dalam loop. Metode ini tidak efisien karena membuang siklus CPU untuk menunggu event yang belum tentu terjadi dan membuat sistem tidak responsif terhadap event mendadak [3]. Untuk mengatasi keterbatasan tersebut, ATmega328P menyediakan mekanisme interrupt hardware yang memungkinkan CPU segera merespons event eksternal tanpa polling, serta modul timer internal untuk penjadwalan periodik yang akurat [5]. Kombinasi interrupt dan timer merupakan fondasi desain sistem embedded real-time, digunakan pada robotika, sistem monitoring, IoT, hingga RTOS [1].

2. STUDI PUSTAKA

García-Tudela dan Marín-Marín [1] menyimpulkan bahwa pendekatan Project-Based Learning adalah strategi paling efektif mengintegrasikan konsep sistem tertanam seperti interrupt dan timer ke dalam kurikulum STEM.

2.1 INTERRUPT DAN ISR

Interrupt adalah mekanisme hardware yang memungkinkan mikrokontroler menghentikan sementara eksekusi program utama untuk menangani suatu kejadian. Ketika interrupt terjadi, CPU menyimpan state program (register dan program counter) ke stack, mengeksekusi ISR (Interrupt Service Routine), kemudian memulihkan state dan melanjutkan program utama [5]. Arduino Uno menyediakan dua pin external interrupt: pin 2 (INT0) dan pin 3 (INT1), yang dikonfigurasi menggunakan attachInterrupt() dengan mode RISING, FALLING,

CHANGE, atau LOW. ISR harus dibuat sesingkat mungkin dan tidak boleh menggunakan delay() maupun Serial.print() [5].

2.2 VARIABEL VOLATILE

Variabel yang dimodifikasi di dalam ISR dan dibaca di program utama harus dideklarasikan dengan keyword volatile. Keyword ini memberitahu compiler untuk tidak meng-cache nilai variabel di register CPU, melainkan selalu membaca nilainya langsung dari RAM. Tanpa volatile, compiler dapat mengoptimasi kode dengan berasumsi nilai variabel tidak berubah di luar alur normal program, yang dapat menyebabkan program utama membaca nilai lama yang sudah tidak valid [4].

2.3 TIMER DAN MILLIS()

ATmega328P memiliki tiga modul timer internal: Timer0 (8-bit), Timer1 (16-bit), dan Timer2 (8-bit). Fungsi millis() memanfaatkan Timer0 untuk menghitung waktu sejak Arduino menyala dalam satuan milidetik. Teknik non-blocking dengan millis() menggunakan pola if (millis() - previousMillis >= interval) untuk memeriksa apakah selang waktu yang ditentukan sudah tercapai tanpa menghentikan eksekusi program [5]. Pendekatan ini memungkinkan sistem menjalankan beberapa task secara pseudo-concurrent dalam satu loop(), berbeda dengan delay() yang memblokir seluruh program selama periode tunggu [4].

3. METODOLOGI

Praktikum ini menggunakan alat dan bahan: Arduino Uno, Breadboard, Push Button, LED (2 buah), Resistor 220Ω (2 buah), Resistor 10kΩ, Kabel Jumper, dan komputer dengan Arduino IDE.

3.1 PERCOBAAN 6A – EXTERNAL INTERRUPT

Push button dihubungkan ke pin 2 dengan konfigurasi INPUT_PULLUP (resistor pull-up internal diaktifkan, sehingga pin membaca HIGH saat tombol tidak ditekan dan LOW saat ditekan). LED dihubungkan ke pin 13. Program mendeklarasikan variabel volatile bool ledState, mendefinisikan ISR tombolInterrupt() yang men-toggle ledState, dan mendaftarkan ISR ke pin 2 dengan mode FALLING menggunakan attachInterrupt(). Pada loop(), nilai ledState langsung ditulis ke pin 13 tanpa perlu polling kondisi tombol.

Tabel 3-1 Konfigurasi Pin Percobaan 6A

No	Komponen	Pin Arduino	Keterangan
1	LED	Pin 13	Output digital
2	Push Button	Pin 2 (INT0)	External interrupt
3	Resistor 10kΩ	GND	Pull-down (opsional)

4	GND	GND	Referensi tegangan
---	-----	-----	--------------------

3.2 PERCOBAAN 6B – TIMER DENGAN MILLIS()

Dua LED dihubungkan ke pin 13 dan pin 12, masing-masing melalui resistor 220Ω ke GND. Program mendefinisikan dua pasang variabel `previousMillis` dan interval untuk setiap LED: LED pertama dengan interval 1000 ms dan LED kedua dengan interval 500 ms. Setiap iterasi `loop()`, nilai `millis()` dibandingkan dengan `previousMillis` masing-masing LED untuk menentukan apakah sudah waktunya toggle, tanpa menggunakan `delay()` sama sekali.

Tabel 3-2 Konfigurasi Pin Percobaan 6B

No	Komponen	Pin Arduino	Keterangan
1	LED 1 (merah)	Pin 13	Interval 1000 ms
2	LED 2 (kuning)	Pin 12	Interval 500 ms
3	Resistor 220Ω (x2)	Seri ke GND	Pembatas arus LED
4	GND	GND	Referensi tegangan

4. HASIL DAN ANALISIS

4.1 PERCOBAAN 6A – EXTERNAL INTERRUPT

Setelah program diunggah, LED berhasil toggle kondisi ON/OFF setiap kali tombol ditekan. Respons LED terasa instan tanpa lag yang terdeteksi. Sistem tidak melakukan polling tombol — `loop()` hanya menulis nilai `ledState` ke pin 13, sementara perubahan `ledState` sepenuhnya ditangani oleh ISR saat interrupt terjadi. Tidak ditemukan bounce yang signifikan karena mode FALLING hanya memicu satu interrupt per penekanan pada transisi tepi turun. Tselegkaridis dan Sapounidis [2] menyatakan bahwa pembelajaran langsung berbasis Arduino meningkatkan pemahaman mahasiswa terhadap mekanisme interrupt hardware secara signifikan.

Jawaban Pertanyaan Praktikum 6A:

- Proses tombol mengubah kondisi LED melalui interrupt: saat tombol ditekan, tegangan pin 2 bertransisi dari HIGH ke LOW (FALLING edge). Hardware interrupt controller ATmega328P mendeteksi transisi ini, menghentikan sementara eksekusi `loop()`, menyimpan state CPU ke stack, lalu memanggil ISR `tombolInterrupt()`. Di dalam ISR, `ledState` di-toggle (`!ledState`). Setelah ISR selesai, CPU memulihkan state dan melanjutkan `loop()` yang kini membaca nilai `ledState` terbaru dan menulisnya ke pin 13.
- Fungsi `attachInterrupt()` mendaftarkan fungsi ISR ke sistem interrupt hardware Arduino. Parameter pertama `digitalPinToInterrupt(2)` mengonversi nomor pin ke nomor interrupt (pin 2 = INT0). Parameter kedua `tombolInterrupt` adalah pointer ke fungsi ISR yang akan dieksekusi. Parameter ketiga FALLING menentukan kondisi pemicu: interrupt hanya aktif saat sinyal bertransisi dari HIGH ke

LOW. Tanpa `attachInterrupt()`, pin 2 tidak akan terhubung ke sistem interrupt hardware.

- ISR tidak disarankan menggunakan `delay()` karena `delay()` bergantung pada Timer0 interrupt untuk menghitung waktu. Selama ISR berjalan, interrupt lain dinonaktifkan (termasuk Timer0), sehingga `delay()` tidak akan pernah selesai dan menyebabkan sistem hang. `Serial.print()` tidak disarankan karena operasi ini membutuhkan buffer yang diisi melalui interrupt UART — memanggil `Serial.print()` di dalam ISR dapat menyebabkan deadlock atau korupsi buffer karena interrupt sedang dinonaktifkan.
- Keyword `volatile` pada `ledState` memberitahu compiler agar tidak meng-cache nilai variabel di register CPU. Tanpa `volatile`, compiler dapat mengoptimasi `loop()` dengan menyimpan nilai `ledState` di register dan tidak membacanya ulang dari RAM setiap iterasi, karena compiler tidak "tahu" bahwa ISR dapat mengubah nilainya kapanpun. Dengan `volatile`, setiap akses ke `ledState` dipaksa membaca langsung dari RAM sehingga nilai selalu terkini.
- Perbedaan mode interrupt: FALLING (digunakan) memicu interrupt saat sinyal turun dari HIGH ke LOW — cocok untuk tombol dengan INPUT_PULLUP karena tombol ditekan menghasilkan FALLING. RISING memicu saat sinyal naik dari LOW ke HIGH — LED toggle saat tombol dilepas, bukan ditekan, terasa kurang natural. CHANGE memicu pada setiap perubahan (naik maupun turun) — LED toggle dua kali per siklus tekan-lepas, blink lebih cepat. LOW memicu terus selama pin LOW — selama tombol ditekan ISR dipanggil berulang kali sangat cepat, LED berkedip tidak terkontrol dan tidak cocok untuk toggle. Kode modifikasi dan penjelasan lengkap terlampir dalam README.md.

4.2 PERCOBAAN 6B – TIMER DENGAN MILLIS()

Kedua LED berhasil berkedip dengan interval yang tepat dan independen: LED pin 13 berkedip setiap 1000 ms dan LED pin 12 berkedip setiap 500 ms tanpa saling mengganggu. Selama LED berkedip, program tetap responsif — tidak ada jeda yang terasa karena tidak ada `delay()` yang memblokir `loop()`. Prabowo dan Irwanto [3] menyatakan bahwa implementasi Arduino dalam pembelajaran berbasis proyek memperdalam pemahaman terhadap manajemen waktu non-blocking pada sistem tertanam.

Jawaban Pertanyaan Praktikum 6B:

- Cara kerja `millis()` pada program: setiap iterasi `loop()`, fungsi `millis()` dipanggil untuk mendapatkan waktu saat ini dalam milidetik sejak Arduino menyala. Nilai ini dikurangi dengan `previousMillis` (waktu terakhir LED di-toggle). Jika selisihnya $\geq$ interval (1000 ms), maka `previousMillis` diperbarui ke waktu saat ini, `ledState` di-toggle, dan nilai baru ditulis ke pin 13. Jika belum mencapai interval, blok `if` dilewati dan `loop()` langsung berulang, memungkinkan kode lain berjalan.

- Perbedaan utama `delay()` dan `millis()`: `delay(1000)` menghentikan seluruh eksekusi program selama 1 detik — CPU terjebak dalam busy-wait dan tidak dapat menjalankan instruksi apapun selama periode tersebut. `millis()` bersifat non-blocking: program terus berjalan dan hanya memeriksa apakah selang waktu sudah tercapai setiap iterasi. Dengan `millis()`, beberapa tugas dengan interval berbeda dapat dijalankan dalam satu `loop()` secara pseudo-concurrent, yang tidak mungkin dilakukan dengan `delay()`.
- `millis()` disebut non-blocking karena tidak menghentikan eksekusi program. Pola `if (millis() - previousMillis >= interval)` hanya melakukan perbandingan aritmatika yang selesai dalam satu siklus clock. Jika kondisi belum terpenuhi, blok `if` dilewati dan `loop()` langsung berulang. CPU selalu bebas untuk mengerjakan instruksi berikutnya, berbeda dengan `delay()` yang memblokir CPU sepenuhnya hingga waktu tunggu habis.
- Modifikasi dua LED dengan interval berbeda: LED pin 13 (1000 ms) dan LED pin 12 (500 ms) menggunakan dua pasang variabel `previousMillis` dan interval yang independen. Kedua LED dikelola dalam satu `loop()` tanpa `delay()`. Penjelasan tiap baris kode terlampir dalam README.md.

4.3 PERTANYAAN ANALISIS

- Keuntungan interrupt dibanding polling: (1) Efisiensi CPU — CPU tidak perlu terus-menerus memeriksa kondisi pin, sehingga bebas mengerjakan tugas lain; (2) Responsivitas tinggi — interrupt merespons event dalam hitungan mikro-detik (satu siklus clock setelah event), sedangkan polling bergantung pada kapan `loop()` mencapai kode pengecekan; (3) Deterministik — waktu respons interrupt dapat diprediksi dan diukur; (4) Hemat daya — pada mode sleep, CPU dapat tidur dan hanya bangun saat interrupt terjadi, menghemat konsumsi energi secara signifikan.
- Timer penting dalam sistem embedded dan real-time karena menyediakan referensi waktu yang presisi dan independen dari eksekusi program. Timer memungkinkan: penjadwalan task periodik yang akurat (misal baca sensor setiap 100 ms); pengukuran durasi event (debounce tombol, pengukuran pulsa); pembangkitan sinyal PWM untuk kontrol motor dan LED; serta implementasi watchdog timer yang mereset sistem jika program macet. Tanpa timer, semua penjadwalan harus dilakukan secara software melalui penghitungan siklus clock yang tidak akurat.
- Alur kerja sistem dengan interrupt dan timer tergabung: `loop()` berjalan terus-menerus menangani tugas rutin seperti pengecekan `millis()` untuk kedipan LED. Ketika tombol ditekan, hardware interrupt memotong eksekusi `loop()`, ISR dieksekusi untuk mengubah state (misal mengaktifkan/menonaktifkan timer), lalu `loop()` dilanjutkan. Timer interrupt (jika digunakan langsung) dapat memicu ISR periodik secara independen dari `loop()`. Keduanya bekerja secara asinkron dan saling melengkapi: interrupt menangani event tidak terjadwal, timer menangani event terjadwal.
- Jika ISR terlalu panjang atau kompleks: (1) Interrupt lain yang datang selama ISR berjalan dapat terlewat karena interrupt dinonaktifkan sementara; (2) Timer0 tidak bertambah selama ISR, menyebabkan `millis()` dan `delay()` kehilangan akurasi waktu; (3) Program utama terhenti terlalu lama, menyebabkan sistem kehilangan responsivitas; (4) Pada sistem RTOS, ISR yang panjang dapat melewati deadline task yang bergantung pada interrupt tersebut. Solusi: ISR hanya meng-set flag (variabel volatile), kemudian proses panjang dilakukan di program utama berdasarkan flag tersebut.

Tabel 4-1 Hasil Percobaan

Percobaan	Parameter	Hasil
6A	Toggle LED via interrupt	Berfungsi sesuai spesifikasi
	Error	Tidak ada
6B	Dua LED non-blocking millis()	1000 ms & 500 ms, responsif
	Error	Tidak ada

5. KESIMPULAN

Praktikum ini telah berhasil mendemonstrasikan implementasi interrupt dan timer pada Arduino Uno. Pada Percobaan 6A, external interrupt berhasil digunakan untuk mengendalikan kondisi LED melalui ISR dengan mode FALLING tanpa polling tombol secara terus-menerus, memberikan respons yang instan dan efisien. Pada Percobaan 6B, teknik `millis()` non-blocking berhasil diimplementasikan untuk mengelola dua LED dengan interval berbeda secara independen dalam satu `loop()` tanpa `delay()`. Dari hasil percobaan disimpulkan bahwa: interrupt meningkatkan responsivitas dan efisiensi CPU secara signifikan dibanding polling; variabel volatile wajib digunakan untuk data yang dimodifikasi di ISR; ISR harus dibuat sesingkat mungkin untuk menghindari kehilangan interrupt lain; serta `millis()` adalah alternatif `delay()` yang memungkinkan multitasking sederhana dalam `loop()` tunggal. Hal ini sejalan dengan García-Tudela dan Marín-Marín [1] bahwa pembelajaran berbasis proyek Arduino meningkatkan pemahaman mahasiswa terhadap sistem tertanam secara menyeluruh.

DAFTAR PUSTAKA

- Garcia-Tudela, P. A. dan Marin-Marín, J.-A., "Use of Arduino in Primary Education: A Systematic Review," *Education Sciences*, Vol. 13, No. 2, hal. 134, 2023. DOI: 10.3390/educsci13020134.
- Tselegkaridis, S. dan Sapounidis, T., "Exploring Students' Hands-On Performance, Attitudes, and Usability with Arduino Modular Boards," *Information*, Vol. 15, No. 2, hal. 88, 2024. DOI: 10.3390/info15020088.
- Prabowo, N. K. dan Irwanto, I., "The Implementation of Arduino Microcontroller Boards in Science: A Bibliometric Analysis from 2008 to 2022," *Journal of Engineering Education Transformations*, Vol. 37, No. 2, hal. 106-123, 2023.

- [4] Mazidi, M. A., Naimi, S., dan Naimi, S., AVR Microcontroller and Embedded Systems: Using Assembly and C, Pearson Education, New Jersey, 2011.
- [5] Arduino, "Arduino Language Reference," <https://docs.arduino.cc/language-reference/>, diakses 13 Mei 2026, 08.00 WIB.